

CI/CD & Deployment Details — DocumentIA

Status: Verificado contra azure-pipelines*.yaml **Audience:** DevOps, SREs, Release Managers

Pipeline Overview

Pipelines Disponibles (5 total)

```
Pipelines en repo root:

azure-pipelines.yaml (Principal: Functions + Admin + AssetResolver)
├ Stages: Build → Migration (disabled) → DeployFunctions → DeployAdmin → DeployAssetResolver → Validate
├ Trigger: Manual (no CI/CD on push)
├ Agents: Windows-latest
└ Artifacts: drop-functions, drop-admin, drop-assetresolver

azure-pipelines-functions.yaml (Functions-only: rápido hotfix)
├ Stages: Build → Deploy
├ Trigger: Manual
└ Target: srbappprodoci

azure-pipelines-admin.yaml (Admin-only: hotfix del Blazor)
├ Stages: BuildAdmin → DeployAdmin
├ Trigger: Manual
└ Target: srbwebadminprodoci

azure-pipelines-assetresolver.yaml (Plugin-only: AssetResolver hotfix)
├ Stages: Build → Deploy (optional)
├ Trigger: Manual
└ Target: srbwebpluginassetresolver

azure-pipelines-bootstrap.yaml (Inicializacion de entorno: permisos, secretos KV, app settings, validacion)
├ Stages: Bootstrap (job unico)
├ Trigger: Manual
└ Requiere: variable group docia-bootstrap-<env>-secrets
```

Current State

- **Nota:** No hay CI/CD automático — todos los pipelines son de ejecución manual
- Los despliegues se parametrizan por `targetEnvironment` (`dev` / `pre` / `prod`); cada entorno tiene su propio Resource Group, Service Connection y Key Vault
- Service Connection por entorno: "AI DocClassExt DEV" / "AI DocClassExt PRE" / "AI DocClassExt PRO"
- .NET 10 Isolated for Functions, .NET 8 for Admin/AssetResolver (SDK 9.x usado solo como host de build/EF tools)

Full Deployment Flow (azure-pipelines.yaml)

Stage 1: Build (Windows-latest)

What Happens:

```
# 1. Use .NET 10 SDK
# 2. Restore NuGet packages
dotnet restore src/backend/DocumentIA.sln

# 3. Build Release configuration
dotnet build src/backend/DocumentIA.sln /p:Configuration=Release

# 4. Run unit tests
dotnet test src/backend/DocumentIA.Tests.Unit/ --configuration Release

# 5. Publish 3 projects
dotnet publish src/backend/DocumentIA.Functions/ -c Release -o drop/functions
dotnet publish src/frontend/DocumentIA.Admin/ -c Release -o drop/admin
dotnet publish src/plugins/DocumentIA.AssetResolver/ -c Release -o drop/assetresolver

# 6. Upload artifacts to pipeline
→ drop-functions (uploaded as artifact)
→ drop-admin (uploaded as artifact)
→ drop-assetresolver (uploaded as artifact)
```

Duration: ~8-12 minutes **Artifacts:** 3 ZIP packages (stored in pipeline cache for deploy stages)

Stage 2: Run Database Migrations

Current Status: DISABLED (condition: false)

This stage does NOT run automatically. To enable:

1. Edit `azure-pipelines.yml`
2. Find: `condition: false` in RunMigrations stage
3. Change to: `condition: true`
4. Commit and re-run

What it does (if enabled):

```
# 1. Install dotnet-ef global tool v9
dotnet tool install --global dotnet-ef --version 9.*

# 2. Apply EF Core migrations
dotnet ef database update `
  --project src/backend/DocumentIA.Data/ `
  --startup-project src/backend/DocumentIA.Functions/ `
  --configuration Release `
  --connection "$(SqlConnectionString)" # From KeyVault

# Result: All new migrations applied to SQL database
```

Alternative: Enable `RunDatabaseMigrationsOnStartup=true` in Functions App Settings → Migrations run automatically on cold start (safer, but adds cold start time)

Stage 3: Deploy Functions (srbappprodocai)

What Happens:

```

# 1. Download drop-functions artifact from Build stage
# 2. Deploy via zipDeploy to Azure Functions App
az functionapp deployment source config-zip `
  --resource-group SRBRGDOCSAIPROD `
  --name srbappprodocai `
  --src artifacts/drop-functions.zip

# 3. Apply 40+ App Settings (post-deploy validation)
# Settings include: CU config, DI config, GPT fallback, timeouts, retries, etc.

# 4. Validate all required CU resiliencia settings are applied
$requiredKeys = @(
  "Extraction__AzureContentUnderstanding__MaxConcurrentCalls",
  "Extraction__AzureContentUnderstanding__HardTimeoutSeconds",
  "Extraction__AzureContentUnderstanding__EnableCircuitBreaker",
  "Extraction__AzureContentUnderstanding__CircuitBreakerFailureThreshold",
  "Extraction__AzureContentUnderstanding__CircuitBreakerOpenSeconds",
  "Extraction__AzureContentUnderstanding__MaxRetries",
  "Extraction__AzureContentUnderstanding__InitialRetryDelayMs",
  "PromptTracing__Enabled",
  "PromptTracing__IncludePromptText",
  "PromptTracing__MaxPromptTextChars"
)

# 5. Fail deployment if any required settings missing
# → Ensures configuration integrity before traffic routed

```

Duration: ~3-5 minutes **Validation:** Automatic (stops deployment if contract violated)

Stage 4: Deploy Admin (srbwebadminprodocai)

```

# 1. Download drop-admin artifact
# 2. Deploy via zipDeploy to App Service
az webapp deployment source config-zip `
  --resource-group SRBRGDOCSAIPROD `
  --name srbwebadminprodocai `
  --src artifacts/drop-admin.zip

# 3. Assign Managed Identity (if not already)
az webapp identity assign `
  --resource-group SRBRGDOCSAIPROD `
  --name srbwebadminprodocai

# 4. Assign RBAC role: "Key Vault Secrets User"
az role assignment create `
  --role "Key Vault Secrets User" `
  --assignee-object-id <principalId> `
  --scope <KeyVaultResourceId>

# 5. Apply Admin App Settings
$adminSettings = @(
  "FunctionsAdminApi__BaseUrl=https://srbappprodocai.azurewebsites.net/api/",
  "FunctionsAdminApi__FunctionKey=@Microsoft.KeyVault(...)"
)

```

Duration: ~2-3 minutes **Depends On:** DeployFunctions must succeed

Stage 5: Deploy AssetResolver (srbwebpluginassetresolver)

```
# 1. Download drop-assetresolver artifact
# 2. Deploy via zipDeploy to App Service
az webapp deployment source config-zip `
  --resource-group SRBRGDOCSAIPROD `
  --name srbwebpluginassetresolver `
  --src artifacts/drop-assetresolver.zip

# 3. Apply AssetResolver App Settings
$assetResolverSettings = @(
  "ConnectionStrings__AssetResolverDb=@Microsoft.KeyVault(...)",
  "ApiKey=@Microsoft.KeyVault(...)"
)
```

Duration: ~2-3 minutes **Depends On:** DeployFunctions must succeed

Stage 6: Validate Configuration Contract

```
# Run validation script to verify all required settings are applied
./scripts/testing/validate-azure-appsettings-contract.ps1 `
  -ResourceGroup SRBRGDOCSAIPROD `
  -FunctionsAppName srbappprodoci `
  -AdminWebAppName srbwebadminprodoci `
  -AssetResolverWebAppName srbwebpluginassetresolver

# Checks:
# All required settings present on Functions App
# All required settings present on Admin App
# All required settings present on AssetResolver
# → Deployment succeeds only if all contracts met
```

Duration: ~1 minute **Gate:** If validation fails, deployment is marked as failed

How to Trigger a Deployment

Via Azure DevOps Portal

1. Go to: https://sareb.visualstudio.com/AI%20DocClassExt/_build
2. Click **Pipelines** (left menu)
3. Select **azure-pipelines** (or specific pipeline: azure-pipelines-functions.yml, etc.)
4. Click **Run pipeline** (top-right button)
5. Verify **Branch:** develop (or main)
6. Click **Run**
7. Monitor: Each stage shows live progress + logs

Via Azure DevOps CLI

```
# Install CLI (if not installed)
# https://learn.microsoft.com/en-us/azure/devops/cli

# Trigger main pipeline
az pipelines run `
  --name "azure-pipelines" `
  --branch develop `
  --project "AI DocClassExt" `
  --organization https://sareb.visualstudio.com
```

Troubleshooting Pipeline Failures

Build Stage Fails

Symptom: `dotnet build failed` or `Test project not found`

Diagnosis Steps:

```
# 1. Check build locally first
cd src/backend/DocumentIA.Functions
dotnet clean
dotnet restore src/backend/DocumentIA.sln
dotnet build src/backend/DocumentIA.sln /p:Configuration=Release

# 2. Check pipeline logs for specific error
# Look for patterns:
# MSB4018: Failed to load ... → Project/package issue
# CS0103: Name not found → Broken reference or missing code
# PackageNotFound → NuGet source or version mismatch
# Tests failed (xUnit/NUnit errors)
# Publish failed → Runtime/framework issue
```

Solutions by Error:

Error	Cause	Fix
<code>MSB4018: GenerateRuntimeConfigurationFiles</code>	Missing/broken .csproj	Verify all project references in .sln
<code>CS0103: 'ClassName' does not exist</code>	Code broken, missing file	Run <code>git status</code> locally, rebuild
<code>NU1101: Unable to find version</code>	NuGet package missing in source	Run <code>dotnet restore --force</code>
<code>Tests failed: 3 failed, 45 passed</code>	Regression in code	Run <code>dotnet test</code> locally, fix failures
<code>Publish to drop failed: Access denied</code>	Pipeline artifact storage issue	Check pipeline run permissions

Recovery Action:

```
# If build fails but code is good locally:
# 1. Hard clean
git clean -fdx
git restore .

# 2. Rebuild
dotnet clean src/backend/DocumentIA.sln
dotnet restore src/backend/DocumentIA.sln /p:Configuration=Release
dotnet build src/backend/DocumentIA.sln /p:Configuration=Release

# 3. If still fails, check .NET version
dotnet --version
# Expected: 10.0.x (for Functions) or 9.0.x (for Admin/AssetResolver)
```

Deploy Stage Fails

Symptom:

- `Deploy to Azure Function App failed`
- `Authentication failed`
- `Connection timeout`
- `Insufficient permissions`

Diagnosis Steps:

```
# 1. Verify service connection
az devops service-endpoint list `
  --project "AI DocClassExt" `
  --query "[].{name:name, type:type, authorization:authorization.scheme}"

# Expected output:
# name: "AI DocClassExt" (service principal connection)
# type: "AzureRM"
# authorization: "ServicePrincipal"

# 2. Check if resource exists
az functionapp show `
  --resource-group SRBRGDOCSAIPROD `
  --name srbappprodoci --query "{name:name, state:state}"

# Expected: name: "srbappprodoci", state: "Running" or "Stopped"

# 3. Verify RBAC permissions (service principal must have Contributor role)
$spId = "INSERT-SERVICE-PRINCIPAL-ID"
az role assignment list `
  --assignee $spId `
  --scope "/subscriptions/YOUR-SUBSCRIPTION-ID/resourceGroups/SRBRGDOCSAIPROD" `
  --query "[].{role:roleDefinitionName, principalName:principalName}"

# Expected: role contains "Contributor" or similar elevated role
```

Solutions by Error:

Error	Cause	Fix
Unauthorized (401)	Service principal expired or deleted	Contact Azure admin, recreate service principal in ADO
Forbidden (403)	Service principal lacks permissions	Assign Contributor role to service principal on resource group
ResourceNotFound (404)	App deleted or resource group wrong	Verify RG: <code>az group show --name SRBRGDOCSAIPROD</code>
Connection timeout	Network/firewall blocking	Check if Function App has IP restrictions; whitelist ADO agents
Artifact not found	Build stage didn't produce artifact	Check Build stage succeeded; verify artifact name in Deploy stage config

Recovery Action:

```
# If deployment fails but app is healthy:
# 1. Verify app is still running
az functionapp show --resource-group SRBRGDOCSAIPROD --name srbappprodoci --query "state"

# 2. Check last deployment
az functionapp deployment list --resource-group SRBRGDOCSAIPROD --name srbappprodoci --max-items 1

# 3. Restart app (sometimes fixes transient issues)
az functionapp restart --resource-group SRBRGDOCSAIPROD --name srbappprodoci

# 4. Retry deployment pipeline
# From ADO: Pipelines → Run → Same build artifact
```

App Settings Validation Fails

Symptom:

- Validation failed: Missing required settings
- Contract validation returned non-zero exit code
- Key Vault access denied

Diagnosis Steps:

```
# 1. Check all current settings
$settings = az functionapp config appsettings list `
  --resource-group SRBRGDOCSAIPROD `
  --name srbappprodocai | ConvertFrom-Json

# Count settings
$settings.Count # Expected: 50-70+

# 2. Look for specific required keys
$required = @(
  "Extraction__AzureContentUnderstanding__MaxConcurrentCalls",
  "Extraction__AzureContentUnderstanding__HardTimeoutSeconds",
  "Extraction__AzureContentUnderstanding__EnableCircuitBreaker",
  "Classification__AzureDocumentIntelligence__Timeout",
  "Classification__Fallback__UseGPT",
  "PromptTracing__Enabled"
)

foreach ($key in $required) {
  $value = $settings | Where-Object { $_.name -eq $key } | Select-Object -ExpandProperty value
  if ($null -eq $value) {
    Write-Error "MISSING: $key"
  } else {
    Write-Output "Found: $key = $value"
  }
}

# 3. Check Key Vault access
az keyvault secret list --vault-name srbkvprodocai --query "length(@)"
# Expected: 40+ secrets stored
```

Solutions:

Issue	Cause	Fix
Missing 10+ settings	Pipeline deploy stage didn't run	Manually re-run "Deploy Functions" stage in pipeline
Key Vault reference fails (@Microsoft.KeyVault)	Managed Identity lacks permission	Assign "Key Vault Secrets User" role to Functions app identity
Invalid Key Vault reference syntax	Typo in secret name	Verify secret exists: <code>az keyvault secret show --vault-name srbkvprodocai --name [name]</code>
Settings present but old values	Cache issue or wrong deployment	Run <code>az functionapp restart</code> then check with <code>az functionapp config appsettings list</code>

Recovery Action:

```
# Manually apply missing settings
$missingSettings = @{
    "Extraction__AzureContentUnderstanding__MaxConcurrentCalls" = "4"
    "Extraction__AzureContentUnderstanding__HardTimeoutSeconds" = "90"
    "PromptTracing__Enabled" = "true"
}

foreach ($key in $missingSettings.Keys) {
    az functionapp config appsettings set `
        --resource-group SRBRGDOCSAIPROD `
        --name srbappprodocai `
        --settings "$key=${$missingSettings[$key]}"
    Write-Output "Set: $key"
}

# Then re-run validation
./scripts/testing/validate-azure-appsettings-contract.ps1 `
    -ResourceGroup SRBRGDOCSAIPROD `
    -FunctionsAppName srbappprodocai
```

Validation Stage Fails

Symptom:

- Validation script failed: Contract mismatch
- One or more required settings missing on Admin app
- AssetResolver app health check failed

Diagnosis:

```
# Run validation manually to see exact failures
./scripts/testing/validate-azure-appsettings-contract.ps1 `
    -ResourceGroup SRBRGDOCSAIPROD `
    -FunctionsAppName srbappprodocai `
    -AdminWebAppName srbwebadminprodocai `
    -AssetResolverWebAppName srbwebpluginassetresolver `
    -Verbose

# Expected output:
# Functions app: 60/60 settings verified
# Admin app: 15/15 settings verified
# AssetResolver app: 8/8 settings verified
# All contracts satisfied

# If fails, will show:
# Functions app: 60/60 (MISSING: Extraction__AzureContentUnderstanding__CircuitBreakerFailureThreshold)
```

Solutions:

```
# Fix missing Admin app settings
az webapp config appsettings set `
  --resource-group SRBRGDOCSAIPROD `
  --name srbwebadminprodoci `
  --settings @(
    "FunctionsAdminApi__BaseUrl=https://srbappprodoci.azurewebsites.net/api/",
    "FunctionsAdminApi__Timeout=30000"
  )

# Fix missing AssetResolver settings
az webapp config appsettings set `
  --resource-group SRBRGDOCSAIPROD `
  --name srbwebpluginassetresolver `
  --settings @(
    "AssetResolver__ApiKey=@Microsoft.KeyVault(VaultName=srbkvprodoci;SecretName=AssetResolverApiKey)"
  )
```

Cold Start Performance Issues

Symptom:

- First request after deploy takes 30+ seconds
- Customer reports timeout in first 5 minutes
- Durable Functions orchestrator doesn't respond immediately

Investigation:

```
# Check Function App logs (live stream)
az functionapp log tail `
  --resource-group SRBRGDOCSAIPROD `
  --name srbappprodoci --provider functionapp

# Look for:
# "Host Status: Running" → Cold start complete
# "Failed to initialize host" → Startup error
# "Timeout waiting for..." → Dependency initialization slow

# Check AppInsights for startup metrics
# Query in AppInsights:
# customMetrics | where name == "DurableTask.Startup" | summarize avg(value) by bin(timestamp, 1m)
```

Solutions:

Issue	Cause	Fix
30-60s cold start	Normal (.NET loading, Durable Task startup)	Expected; use warmup requests or keep-alive
90s+ cold start	Key Vault or database connection slow	Check Key Vault latency, database connectivity
Repeated cold starts	App is crashing/restarting	Check logs for exceptions; check memory/CPU
Durable Functions not responding	Hub instance not started	Run warmup trigger manually: <code>curl https://srbappprodoci.azurewebsites.net/api/health</code>

Recovery:

```
# 1. Warm up the app (trigger a lightweight function)
curl -X GET "https://srbappprodcai.azurewebsites.net/api/health" -Headers @{x-functions-key="$FunctionKey"}

# 2. If that fails, restart
az functionapp restart --resource-group SRBRGDOCSAIPROD --name srbappprodcai

# 3. Monitor startup
az functionapp log tail --resource-group SRBRGDOCSAIPROD --name srbappprodcai
```

Rollback Procedures

Option 1: Re-run Previous Successful Deployment (Fastest — Recommended)

Best for: Non-breaking issues, need to quickly revert to known good version

Steps (5-10 minutes):

```
# 1. Find previous successful run
az pipelines runs list `
  --pipeline-ids "PIPELINE-ID" `
  --status "completed" `
  --result "succeeded" `
  --top 5 `
  --query "[].{id:id, status:status, result:result, createdAt:createdDate}"

# Output example:
# id: 12345, status: "completed", result: "succeeded", createdAt: "2026-06-09T14:30:00Z"
# id: 12344, status: "completed", result: "succeeded", createdAt: "2026-06-08T03:15:00Z"

# 2. Note the run ID of the version you want to revert to (e.g., 12344)

# 3. Go to Azure DevOps UI:
# → Pipelines → Runs → Select run 12344
# → Click "Rerun" → "Rerun from Stage"
# → Select "DeployFunctions" (or specific failed stage)
# → Confirm

# 4. Monitor deployment
az pipelines runs show `
  --id 12346 ` # New re-run ID
  --query "{status:status, result:result, createdAt:createdDate}"
```

Pros: Low risk, tested artifact, fast **Cons:** Must have previous successful run **Time:** 5-10 min **Risk Level:** Low

Option 2: Immediate Database Rollback + Code Revert (For Data Issues)

Best for: Database migration failed, data corruption, need both code AND data rollback

Steps (15-20 minutes, requires downtime):

```

# 1. STOP all functions apps immediately
az functionapp stop --resource-group SRBRGDOCSAIPROD --name srbappprodocai
az webapp stop --resource-group SRBRGDOCSAIPROD --name srbwebadminprodocai

Write-Output "Apps stopped at $(Get-Date -Format 'HH:mm:ss')"
```

2. CREATE database backup of current (corrupted) state

```

az sql db backup create `
--server "YOUR-SQL-SERVER" `
--database "DocumentIA" `
--resource-group SRBRGDOCSAIPROD

Write-Output "Backup created (preserve for investigation)"
```

3. RESTORE database to pre-deployment backup

Identify backup timestamp from before failed deploy

```

$backups = az sql db backup list `
--server "YOUR-SQL-SERVER" `
--database "DocumentIA" `
--resource-group SRBRGDOCSAIPROD `
--query "[].[name, earliestRestoreDate, createdDate]" -o json | ConvertFrom-Json

# Show backups
$backups | Format-Table
```

Restore to backup taken before deployment

Example: "2026-06-08T02:00:00Z" (day before issue)

```

az sql db restore `
--server "YOUR-SQL-SERVER" `
--database "DocumentIA" `
--backup-name "SQLPool-2026-06-08T02-00-00Z" `
--resource-group SRBRGDOCSAIPROD `
--dest-database "DocumentIA" `
--subscription "YOUR-SUBSCRIPTION"

Write-Output "Database restored to pre-deployment state"
```

4. REVERT code to previous version

```

git checkout v1.4.1 # or whatever previous tag
Write-Output "Code reverted to v1.4.1"
```

5. RE-RUN deployment with PREVIOUS version

Via ADO: Pipelines → Run → Same flow, new artifact built from reverted code

6. RESTART apps

```

az functionapp start --resource-group SRBRGDOCSAIPROD --name srbappprodocai
az webapp start --resource-group SRBRGDOCSAIPROD --name srbwebadminprodocai

Write-Output "Apps restarted at $(Get-Date -Format 'HH:mm:ss')"
```

7. VERIFY

```

./tests/smoke_e2e.ps1 -Environment Production

Write-Output "Rollback complete – Monitoring..."
```

Pros: Comprehensive, handles data issues, documented backup preserved **Cons:** Causes downtime (apps stopped for 10-15 min), requires DB restore **Time:** 15-20 min **Risk Level:** Medium (affects users)

Option 3: Slot Swap (Zero-Downtime, Requires Setup)

Prerequisites: Must have staging slot pre-configured (requires manual setup first time)

Setup (one-time, ~5 minutes):

```
# Create staging slot for Functions App
az functionapp deployment slot create `
  --resource-group SRBRGDOCSAIPROD `
  --name srbappprodocai `
  --slot staging

# Create staging slot for Admin App
az webapp deployment slot create `
  --resource-group SRBRGDOCSAIPROD `
  --name srbwebadminprodocai `
  --slot staging
```

During Deployment:

- Deploy new version to **staging** slot (not production)
- Run smoke tests on staging
- If tests pass: Swap production ↔ staging (instant)
- If tests fail: Just delete staging, keep production running

Rollback via Slot Swap (30 seconds):

```
# Swap back to previous version (still in production slot after swap)
az functionapp deployment slot swap `
  --resource-group SRBRGDOCSAIPROD `
  --name srbappprodocai `
  --slot staging

# Do same for Admin
az webapp deployment slot swap `
  --resource-group SRBRGDOCSAIPROD `
  --name srbwebadminprodocai `
  --slot staging

Write-Output "Swapped back in 30 seconds!"
```

Pros: Zero downtime, instant rollback (30 sec), safest approach **Cons:** Requires staging slot setup beforehand, higher resource costs **Time:** 30 seconds **Risk Level:** Very Low

Quick Decision Tree

```
Rollback needed?
|
├─ Bug in code only (no data changes)
|   └─ Option 1: Re-run previous deployment  FASTEST
|
├─ Database migration failed / Data corrupted
|   └─ Option 2: Stop → Restore DB → Revert code  DOWNTIME
|
└─ Have staging slots configured
    └─ Option 3: Swap slots  ZERO-DOWNTIME
```

Secrets Management

Location: Azure Key Vault (srbkvprodocai)

All sensitive data stored in KeyVault; referenced in App Settings as:

```
@Microsoft.KeyVault(VaultName=srbkvprodocai;SecretName=SecretName)
```

Secret Naming Convention:

```
Extraction--AzureContentUnderstanding--ApiKey (maps to Extraction:AzureContentUnderstanding:ApiKey)
Classification--AzureDocumentIntelligence--ApiKey
Database--ConnectionString
GDC--AuthToken
```

How to Rotate a Secret

```
# 1. Update secret in KeyVault
az keyvault secret set `
  --vault-name srbkvprodoci `
  --name "Extraction--AzureContentUnderstanding--ApiKey" `
  --value "new-api-key-value"

Write-Output "Secret rotated in Key Vault"

# 2. Apps pick it up automatically within 5-10 minutes
# (Each request checks KeyVault for latest value)
# No restart needed!

# 3. Verify the new value is there
az keyvault secret show `
  --vault-name srbkvprodoci `
  --name "Extraction--AzureContentUnderstanding--ApiKey" `
  --query "value" -o tsv
```

Pro Tip: Rotate API keys every 90 days (set calendar reminder)

Real-Time Monitoring During Deployment

Live Dashboard (Recommended)

Open in browser immediately after starting deployment:

```
AppInsights: https://portal.azure.com → Search "srbappprodoci" → Live Metrics Stream
```

Watch for (in order):

1. Server Response Time (should be steady, not increasing)
2. Requests/sec (should not drop to 0)
3. Failed Requests (should stay < 1%)
4. Custom Metrics: `DocumentIA.Duracion.Total` (should stay < 60s P95)

Post-Deployment Validation (Detailed)

```

# Apps Running?
$functions = az functionapp show `
  --resource-group SRBRGDOCSAIPROD `
  --name srbappprodocai `
  --query "state"

$admin = az webapp show `
  --resource-group SRBRGDOCSAIPROD `
  --name srbwebadminprodocai `
  --query "state"

if ($functions -eq "Running" -and $admin -eq "Running") {
  Write-Output "All apps running"
} else {
  Write-Output "App not running: Functions=$functions, Admin=$admin"
}

# Application Insights Errors?
$errors = az monitor metrics list `
  --resource "srbappprodocai" `
  --resource-group SRBRGDOCSAIPROD `
  --metric "Exceptions" `
  --start-time "$(Get-Date -AsUTC -Format 'yyyy-MM-ddTHH:00:00Z')" `
  --interval PT1M `
  --query "value[0].timeseries[0].data | length(@)"

if ($errors -lt 3) {
  Write-Output "Error rate acceptable ($errors exceptions in last hour)"
} else {
  Write-Output "Elevated error rate ($errors exceptions)"
}

# Performance OK?
$latency = az monitor metrics list `
  --resource "srbappprodocai" `
  --resource-group SRBRGDOCSAIPROD `
  --metric "ServerResponseTime" `
  --start-time "$(Get-Date -AsUTC -Format 'yyyy-MM-ddTHH:00:00Z')" `
  --query "value[0].timeseries[0].data[-1].average"

if ($latency -lt 2000) { # 2000ms = 2s
  Write-Output "Response time acceptable ($latency ms)"
} else {
  Write-Output "Elevated latency ($latency ms)"
}

# Smoke Test
./tests/smoke_e2e.ps1 -Environment Production
# Should complete with All tests passed

# Run validation contract
./scripts/testing/validate-azure-appsettings-contract.ps1 `
  -ResourceGroup SRBRGDOCSAIPROD `
  -FunctionsAppName srbappprodocai `
  -AdminWebAppName srbwebadminprodocai `
  -AssetResolverWebAppName srbwebpluginassetresolver

Write-Output "Deployment validation complete"

```

Deployment Checklist (Pre-Deployment)

Run 30 minutes BEFORE triggering pipeline:

- ☐ **Code Changes Reviewed**
 - ☐ All PRs merged and approved (≥2 reviewers for major, ≥1 for minor)

- ☐ `git log --oneline -10` shows expected commits
- ☐ No WIP (work-in-progress) markers in code
- ☐ **Tests Passing**
 - ☐ Unit tests: `dotnet test --configuration Release` → All green
 - ☐ Integration tests: All pass locally
 - ☐ No flaky tests that might fail in CI
- ☐ **Code Quality**
 - ☐ Code analysis: `dotnet build /p:EnforceCodeStyleInBuild=true` → No errors
 - ☐ No NuGet security vulnerabilities: `dotnet list package --vulnerable` → Empty
 - ☐ .NET version correct: `dotnet --version` → 10.x or 9.x
- ☐ **Database Ready (if migrations planned)**
 - ☐ Backup taken recently (< 1 week): `az sql db backup list --server ... --database DocumentIA --top 1`
 - ☐ Migration script tested locally: `dotnet ef database update`
 - ☐ No pending migrations: `dotnet ef migrations list --startup-project ... | grep "Pending"`
- ☐ **Infrastructure Capacity**
 - ☐ CPU < 70%: `az monitor metrics list --resource ... --metric "CpuPercentage"`
 - ☐ Memory < 70%: `az monitor metrics list --resource ... --metric "MemoryPercentage"`
 - ☐ Storage adequate: `az storage account show --name ... --query "primaryEndpoints"`
 - ☐ Key Vault secrets valid (not expired): `az keyvault secret list --vault-name srbkvprodcai --query "[?attributes.expires < now_add('7d')].id" | Should be empty`
- ☐ **Monitoring Configured**
 - ☐ Application Insights connected: `az functionapp config app get-values --resource-group SRBRGDOCSAIPROD --name srbappprodcai | grep APPINSIGHTS`
 - ☐ Alerts set up (high error rate, latency spikes, etc.)
- ☐ **Communication Sent**
 - ☐ Slack/email notification sent to team
 - ☐ Customer support notified (if applicable)
 - ☐ Message includes expected start time and duration
- ☐ **Approval Obtained**
 - ☐ Tech Lead signed off
 - ☐ Product Owner acknowledged (for major releases)

Real-Time Deployment Timeline

Typical deployment takes 45-60 minutes. Expected milestones:

```
T+00 min: Deploy button clicked in ADO
├ Pipeline started
|
T+02 min: Build stage running
├ dotnet restore...
├ dotnet build...
├ dotnet test...
|
T+12 min: Build complete, publishing
├ drop-functions artifact created
├ drop-admin artifact created
├ drop-assetresolver artifact created
|
T+14 min: Deploy Functions stage
├ Downloading artifact
├ Running: az functionapp deployment source config-zip
├ Applying 60+ app settings
├ Validating configuration contract
|
T+20 min: Deploy Admin stage
├ Downloading artifact
├ Deploying to srbwebadminprodocai
├ Assigning Managed Identity
├ Applying RBAC roles
|
T+25 min: Deploy AssetResolver stage
├ Downloading artifact
├ Deploying to srbwebpluginassetresolver
├ Setting up plugin configuration
|
T+30 min: Validate stage
├ Running: validate-azure-appsettings-contract.ps1
├ Checking Functions app settings (60/60)
├ Checking Admin app settings (15/15)
├ Checking AssetResolver settings (8/8)
|
T+32 min: Deployment complete
|
T+35 min: Post-deployment checks
├ Smoke tests running
├ Checking AppInsights for errors
├ Monitoring latency

T+40+ min: Stable, monitoring continues
```

Common Deployment Scenarios

Scenario 1: Standard Release (2-week sprint cycle)

1. Create PR on develop branch
2. Code review (24h)
3. Merge to develop
4. Create release/v1.5.0 branch
5. Update version numbers
6. Run azure-pipelines.yml → All stages pass
7. Monitor 24+ hours
8. Declare release stable
9. Tag: git tag v1.5.0
10. Merge release branch back to develop

Time: 1-2 weeks **Risk:** Low **Approval:** Tech Lead

Scenario 2: Urgent Hotfix (Critical P1 bug)

1. Create hotfix/v1.4.2.1 branch from v1.4.2 tag
2. Apply minimal fix (3-5 lines)
3. Quick review (1 senior developer)
4. Run azure-pipelines-functions.yml → Fast path
5. Monitor 4+ hours
6. If stable: Tag v1.4.2.1, close ticket
7. If issues: Auto-rollback to v1.4.2 (Option 1)

Time: 4-8 hours **Risk:** Medium **Approval:** CTO + Tech Lead (expedited)

Scenario 3: Configuration-Only Change

1. No code change, just settings update
2. Manually update Key Vault secrets
3. NO pipeline run needed (apps reload secrets automatically)
4. Verify: az functionapp config appsettings list ...
5. Monitor for 30+ minutes

Time: 5-10 minutes **Risk:** Low **Approval:** Tech Lead

Monitoring Queries (KQL in AppInsights)

Copy-paste ready queries:

1. Error Rate (Last Hour)

```
customMetrics
| where timestamp > ago(1h)
| extend isError = iif(name contains "Error" or name contains "Failure", 1, 0)
| summarize total=count(), errors=sum(isError) by bin(timestamp, 5m)
| extend errorPct = round(100.0 * errors / total, 2)
| project timestamp, total, errors, errorPct
```

2. P95 Latency (Last Hour)

```
customMetrics
| where timestamp > ago(1h)
| where name == "DocumentIA.Duracion.Total"
| summarize p50=percentile(value, 50), p95=percentile(value, 95), p99=percentile(value, 99) by bin(timestamp, 5m)
| project timestamp, p50, p95, p99
```

3. CU Circuit Breaker Status

```
customMetrics
| where timestamp > ago(1h)
| where name in ("CU.CircuitOpen", "CU.CircuitClosed", "CU.CircuitFailover")
| summarize count() by name, tostring(customDimensions.Reason)
```

4. Document Processing Summary

```

customMetrics
| where timestamp > ago(1h)
| where name == "DocumentProcessed"
| summarize
    total=count(),
    success=sum(iff(customDimensions.EstadoFinal == "SUCCESS", 1, 0)),
    error=sum(iff(customDimensions.EstadoFinal == "ERROR", 1, 0))
| extend successPct=round(100.0 * success / total, 1)

```

Emergency Contact & Escalation

On-Call During Deployment:

Role	Contact	When
Deployment Lead	Responsable del despliegue en curso	Started or on-going deployment
Tech Lead	Responsable técnico	Deployment fails, needs decision
Responsable de operaciones / dirección técnica	Escalado de decisión	P1 issue, rollback decision required
Administración de base de datos	Responsable de BD	Database issues, restore needed

Canal de incidencias: #incidentes-produccion

Appendix: Quick Commands Reference

```

# ===== DEPLOYMENT CONTROL =====
# Trigger pipeline
az pipelines run --name azure-pipelines --branch develop --project "AI DocClassExt"

# Check deployment status
az pipelines runs show --id <RUN-ID> --query "{status:status, result:result}"

# Get last 5 deployments
az pipelines runs list --pipeline-ids <PIPELINE-ID> --top 5

# ===== VERIFICATION =====
# Check Functions App
az functionapp show --resource-group SRBRGDOCSAIPROD --name srbappprodocai --query "{name:name, state:state}"

# View all app settings
az functionapp config appsettings list --resource-group SRBRGDOCSAIPROD --name srbappprodocai

# Stream logs
az functionapp log tail --resource-group SRBRGDOCSAIPROD --name srbappprodocai

# ===== TROUBLESHOOTING =====
# Restart app
az functionapp restart --resource-group SRBRGDOCSAIPROD --name srbappprodocai

# Restore database
az sql db restore --server YOUR-SERVER --database DocumentIA --backup-name ... --resource-group SRBRGDOCSAIPROD

# Rotate secret
az keyvault secret set --vault-name srbkvprodocai --name "KeyName" --value "new-value"

# Run validation
./scripts/testing/validate-azure-appsettings-contract.ps1 -ResourceGroup SRBRGDOCSAIPROD -FunctionsAppName srbappprodocai

```

Related Docs:

- [INFRAESTRUCTURA_REAL_DESPLEGADA.md](#) — Full infrastructure topology
- [TROUBLESHOOTING_DIAGNOSTICO.md](#) — Diagnostic procedures
- [PERFORMANCE_TUNING.md](#) — Configuration tuning